

COM 321 OPERATING SYSTEM II

1.0 Memory Management Techniques

Memory management function of operating system helps in allocating the main memory space to the processes and their data at the time of their execution.

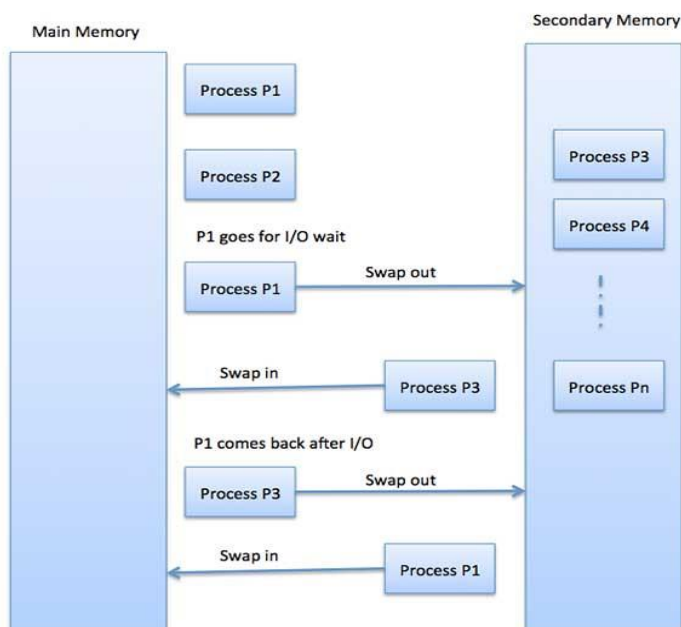
The following are the three key memory management techniques used by an operating system:

1. Swapping
2. Paging
3. Segmentation

1.1 Swapping

Swapping is mechanisms in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes. At some later time, the system swaps back the process from the secondary storage to main memory.

Though performance is usually affected by swapping process but it helps in running multiple and big processes in parallel and that's the reason **Swapping is also known as a technique for memory compaction.**



The total time taken by swapping process includes the time it takes to move the entire process to a secondary disk and then to copy the process back to memory, as well as the time the process takes to regain main memory.

Let us assume that the user process is of size 2048KB and on a standard hard disk where swapping will take place has a data transfer rate around 1 MB per second. The actual transfer of the 1000K process to or from memory will take

$$\begin{aligned} & 2048\text{KB} / 1024\text{KB per second} \\ & = 2 \text{ seconds} \\ & = 2000 \text{ milliseconds} \end{aligned}$$

Now considering in and out time, it will take complete 4000 milliseconds plus other overhead where the process competes to regain main memory.

Fixed Partitioning

The earliest and one of the simplest technique which can be used to load more than one processes into the main memory is Fixed partitioning or Contiguous memory allocation. In this technique, the main memory is divided into partitions of equal or different sizes. The operating system always resides in the first partition while the other partitions can be used to store user processes. The memory is assigned to the processes in contiguous way.

In fixed partitioning,

1. The partitions cannot overlap.
2. A process must be contiguously present in a partition for the execution.

There are various cons of using this technique.

1. Internal Fragmentation

If the size of the process is lesser than the total size of the partition then some size of the partition get wasted and remain unused. This is wastage of the memory and called internal fragmentation.

As shown in the image below, the 4 MB partition is used to load only 3 MB process and the remaining 1 MB got wasted.

2. External Fragmentation

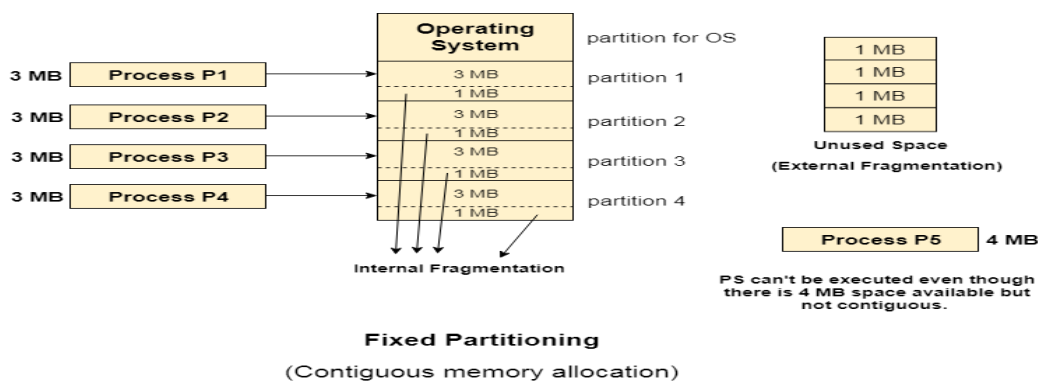
The total unused space of various partitions cannot be used to load the processes even though there is space available but not in the contiguous form. As shown in the image below, the remaining 1 MB space of each partition cannot be used as a unit to store a 4 MB process. Despite of the fact that the sufficient space is available to load the process, process will not be loaded.

3. Limitation on the size of the process

If the process size is larger than the size of maximum sized partition then that process cannot be loaded into the memory. Therefore, a limitation can be imposed on the process size that is it cannot be larger than the size of the largest partition.

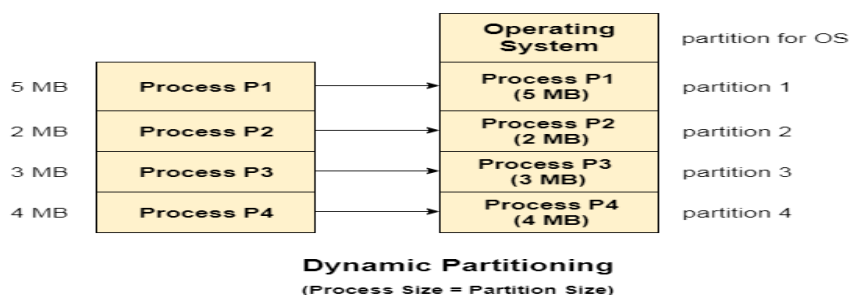
4. Degree of multiprogramming is less

By Degree of multi programming, we simply mean the maximum number of processes that can be loaded into the memory at the same time. In fixed partitioning, the degree of multiprogramming is fixed and very less due to the fact that the size of the partition cannot be varied according to the size of processes.



Dynamic Partitioning

Dynamic partitioning tries to overcome the problems caused by fixed partitioning. In this technique, the partition size is not declared initially. It is declared at the time of process loading. The first partition is reserved for the operating system. The remaining space is divided into parts. The size of each partition will be equal to the size of the process. The partition size varies according to the need of the process so that the internal fragmentation can be avoided.



Advantages of Dynamic Partitioning over fixed partitioning

1. No Internal Fragmentation

Given the fact that the partitions in dynamic partitioning are created according to the need of the process, It is clear that there will not be any internal fragmentation because there will not be any unused remaining space in the partition.

2. No Limitation on the size of the process

In Fixed partitioning, the process with the size greater than the size of the largest partition could not be executed due to the lack of sufficient contiguous memory. Here, In Dynamic partitioning, the process size can't be restricted since the partition size is decided according to the process size.

3. Degree of multiprogramming is dynamic

Due to the absence of internal fragmentation, there will not be any unused space in the partition hence more processes can be loaded in the memory at the same time.

Disadvantages of dynamic partitioning

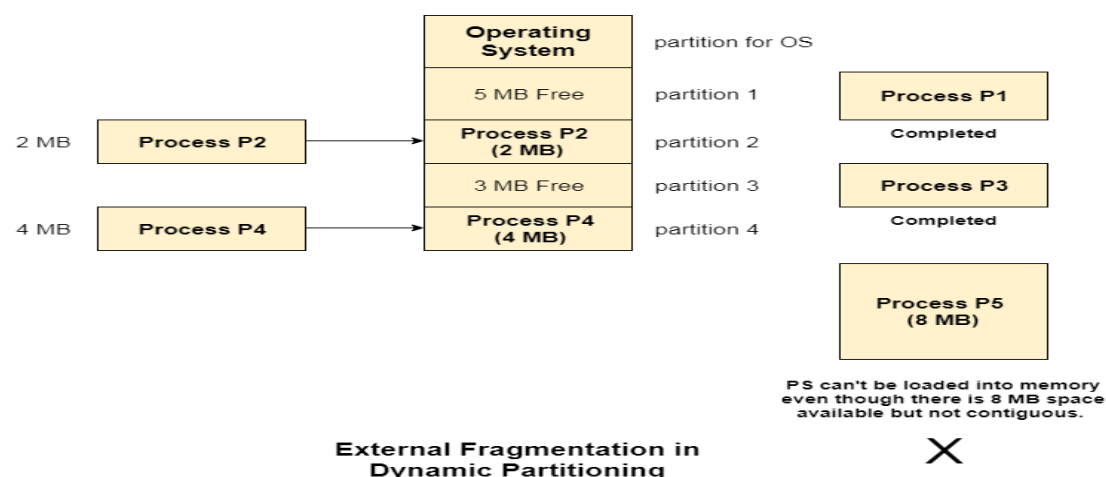
External Fragmentation

Absence of internal fragmentation doesn't mean that there will not be external fragmentation.

Let's consider three processes P1 (1 MB) and P2 (3 MB) and P3 (1 MB) are being loaded in the respective partitions of the main memory.

After some time P1 and P3 got completed and their assigned space is freed. Now there are two unused partitions (1 MB and 1 MB) available in the main memory but they cannot be used to load a 2 MB process in the memory since they are not contiguously located.

The rule says that the process must be contiguously present in the main memory to get executed. We need to change this rule to avoid external fragmentation.

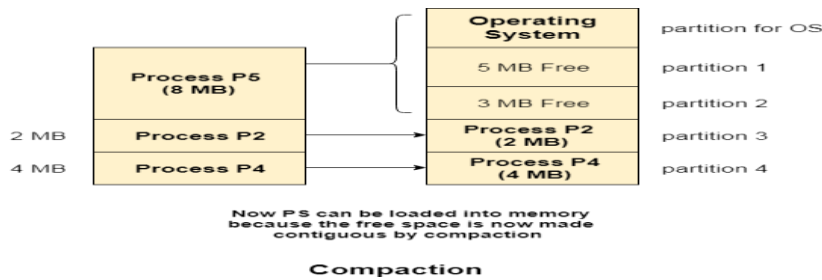


Complex Memory Allocation

In Fixed partitioning, the list of partitions is made once and will never change but in dynamic partitioning, the allocation and deallocation is very complex since the partition size will be varied every time when it is assigned to a new process. OS has to keep track of all the partitions. Due to the fact that the allocation and deallocation are done very frequently in dynamic memory allocation and the partition size will be changed at each time, it is going to be very difficult for OS to manage everything.

Compaction

We got to know that the dynamic partitioning suffers from external fragmentation. However, this can cause some serious problems. To avoid compaction, we need to change the rule which says that the process can't be stored in the different places in the memory. We can also use compaction to minimize the probability of external fragmentation. In compaction, all the free partitions are made contiguous and all the loaded partitions are brought together. By applying this technique, we can store the bigger processes in the memory. The free partitions are merged which can now be allocated according to the needs of new processes. This technique is also called defragmentation.



As shown in the image above, the process P5, which could not be loaded into the memory due to the lack of contiguous space, can be loaded now in the memory since the free partitions are made contiguous.

Problem with Compaction

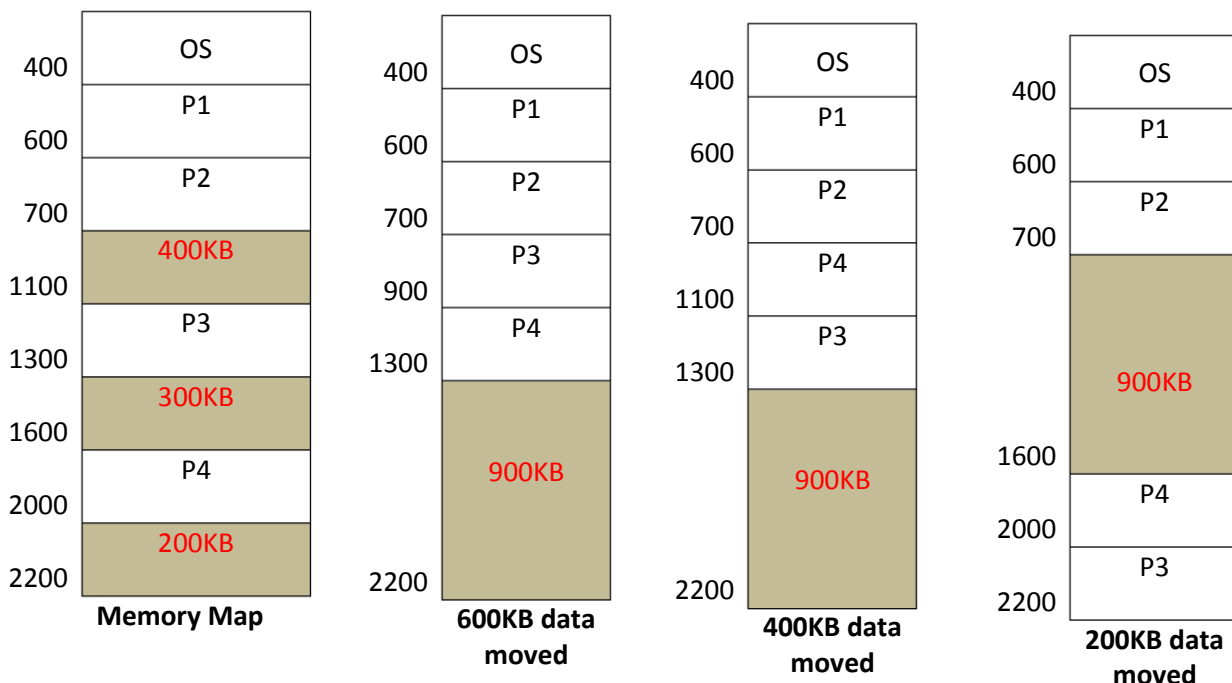
- The efficiency of the system is decreased in the case of compaction due to the fact that all the free spaces will be transferred from several places to a single place.
- Huge amount of time is invested for this procedure and the CPU will remain idle for all this time. Despite of the fact that the compaction avoids external fragmentation, it makes system inefficient.

Let us consider that OS needs 6 NS to copy 1 byte from one place to another.

1. 1 B transfer needs 6 NS
2. 256 MB transfer needs $256 \times 2^{20} \times 6 \times 10^{-9}$ secs

Hence, it is proved to some extent that the larger size memory transfer needs some huge amount of time that is in seconds.

Example: What is the minimum data move when performing compaction on the given memory



Disadvantage of Dynamic Partitioning

The main disadvantage of Dynamic Partitioning is External fragmentation. Although, this can be removed by Compaction but as we have discussed earlier, the compaction makes the system inefficient.

We need to find out a mechanism which can load the processes in the partitions in a more optimal way. Let us discuss a dynamic and flexible mechanism called paging.

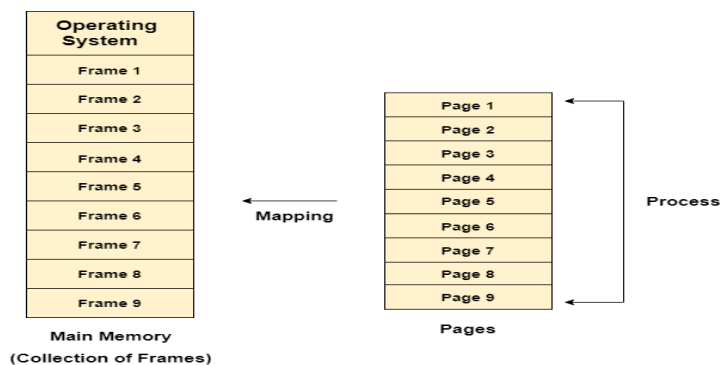
1.2 Paging

Let's consider a process P1 of size 2 MB and the main memory which is divided into three partitions. Out of the three partitions, two partitions are holes of size 1 MB each. P1 needs 2 MB space in the main memory to be loaded. We have two holes of 1 MB each but they are not contiguous. Although, there is 2 MB space available in the main memory in the form of those holes but that remains useless until it becomes contiguous. This is a serious problem to address. We need to have some kind of mechanism which can store one process at different locations of the memory. The idea behind paging is to divide the process in pages so that, we can store them in the memory at different holes.

Paging with Example

In Operating Systems, Paging is a storage mechanism used to retrieve processes from the secondary storage into the main memory in the form of pages. The main idea behind the paging is to divide each process in the form of pages. The main memory will also be divided in the form of frames. One page of the process is to be stored in one of the frames of the memory. The pages can be stored at the different locations of the memory but the priority is always to find the contiguous frames or holes. Pages of the process are brought into the main memory only when they are required otherwise they reside in the secondary storage.

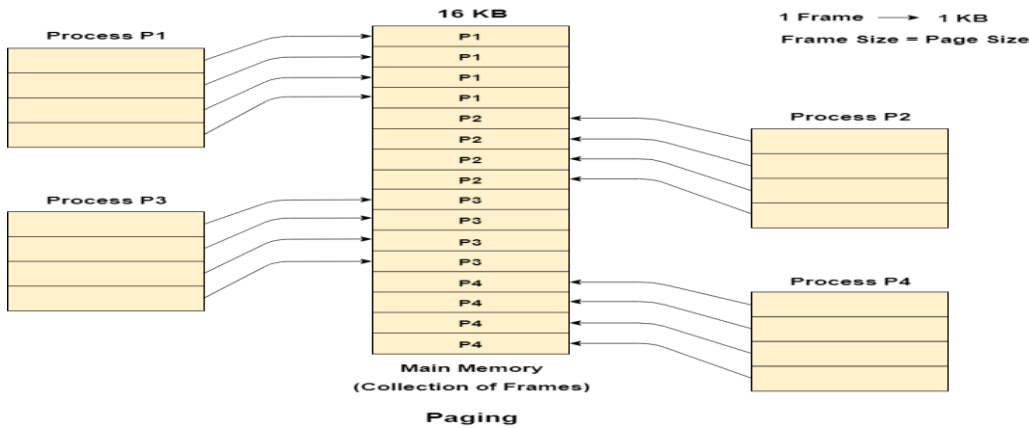
Different operating systems define different frame sizes. The sizes of each frame must be equal. Considering the fact that the pages are mapped to the frames in Paging, page size needs to be as same as frame size.



Example

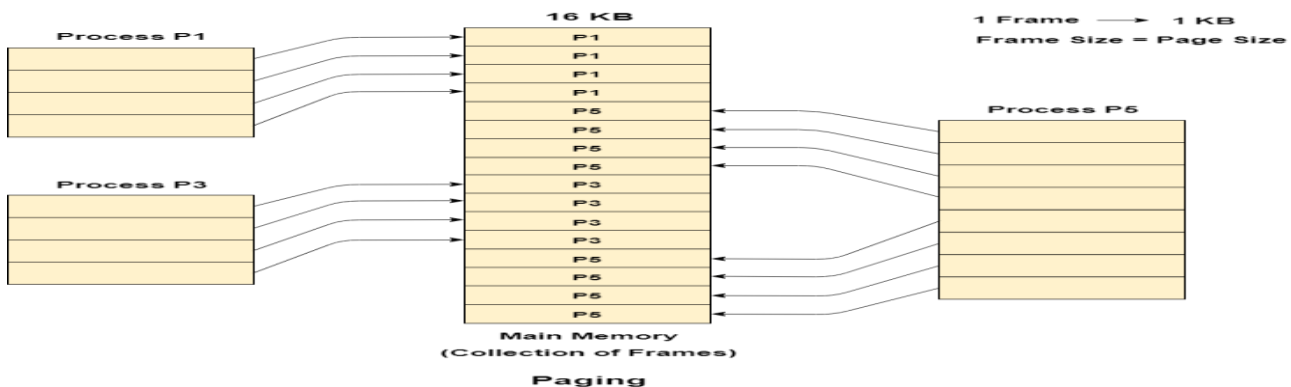
Let us consider the main memory size 16 KB and Frame size is 1 KB therefore the main memory will be divided into the collection of 16 frames of 1 KB each. There are 4 processes in the system that is P1, P2, P3 and P4 of 4 KB each. Each process is divided into pages of 1 KB each so that one page can be stored in one frame. Initially, all the frames are empty therefore pages of the processes will get stored in the contiguous way.

Frames, pages and the mapping between the two are shown in the image below.



Let us consider that, P2 and P4 are moved to waiting state after some time. Now, 8 frames become empty and therefore other pages can be loaded in that empty place. The process P5 of size 8 KB (8 pages) is waiting inside the ready queue.

Given the fact that, we have 8 noncontiguous frames available in the memory and paging provides the flexibility of storing the process at the different places. Therefore, we can load the pages of process P5 in the place of P2 and P4.



Physical and Logical Address Space

Memory unit is a physical address. Logical address is also known as a Virtual address. Virtual and physical addresses are the same in compile-time and load-time address-binding schemes. Virtual and physical addresses differ in execution-time address-binding scheme.

The set of all logical addresses generated by a program is referred to as a logical address space. The set of all physical addresses corresponding to these logical addresses is referred to as a physical address space.

The run-time mapping from virtual to physical address is done by the memory management unit (MMU) which is a hardware device. MMU uses following mechanism to convert virtual address to physical address.

- The value in the base register is added to every address generated by a user process which is treated as offset at the time it is sent to memory. For example, if the base register value is 10000, then an attempt by the user to use address location 100 will be dynamically reallocated to location 10100.
- The user program deals with virtual addresses; it never sees the real physical addresses.

Effective access Time

Each operating system has its own methods for storing page tables. Most allocate a page table for each process. A pointer to the page table is stored with the other register values (like the instruction counter) in the process control block. The hardware implementation of the page table can be done in several ways. In the simplest case, the page table is implemented as a set of dedicated registers. These registers should be built with very high-speed logic to

make the paging-address translation efficient. The use of registers for the page table is satisfactory if the page table is reasonably small (for example, 256 entries).

The standard solution to this problem is to use a special, small, fast lookup hardware cache, called translation look-aside buffer (TLB). The TLB is associative, high speed memory. Each entry in the TLB consists of two parts: a key (or tag) and a value. When the associative memory is presented with an item, it is compared with all keys simultaneously. If the item is found, the corresponding value field is returned. The search is fast; the hardware, however, is expensive. Typically, the number of entries in a TLB is small, often numbering between 64 and 1,024.

The TLB is used with page tables in the following way. The TLB contains only a few of the page-table entries. When a logical address is generated by the CPU, its page number is presented to the TLB. If the page number is found, its frame number is immediately available and is used to access memory. The whole task may take less than 10 percent longer than it would if an unmapped memory reference were used.

If the page number is not in the TLB (known as a TLB miss), a memory reference to the page table must be made. When the frame number is obtained, we can use it to access memory (Figure below). In addition, we add the page number and frame number to the TLB, so that they will be found quickly on the next reference. If the TLB is already full of entries, the operating system must select one for replacement. Replacement policies range from least recently used (LRU) to random. Furthermore, some TLBs allow entries to be wired down, meaning that they cannot be removed from the TLB. Typically, TLB entries for kernel code are often wired down.

The percentage of times that a particular page number is found in the TLB is called the hit ratio. An 80-percent hit ratio means that we find the desired page number in the TLB 80 percent of the time. If it takes 20 nanoseconds to search the TLB, and 100 nanoseconds to access memory, then a mapped memory access takes 120 nanoseconds when the page number is in the TLB.

If we fail to find the page number in the TLB (20 nanoseconds), then we must first access memory for the page table and frame number (100 nanoseconds), and then access the desired byte in memory (100 nanoseconds), for a total of 220 nanoseconds. To find the effective memory-access time, we must weigh each case by its probability:

$$\begin{aligned} EAT(\text{effective access time}) &= P * \text{hit memory time} + (1-P) * \text{miss memory time.} \\ \text{effective access time} &= 0.80 * 120 + 0.20 * 220 \\ &= 140 \text{ nanoseconds.} \end{aligned}$$

In this example, we suffer a 40-percent slowdown in memory access time (from 100 to 140 nanoseconds).

Example: Main memory and TLB are having accessing time of 220ns and 120ns respectively. The TLB hit ratio is 98%. Calculate the Effective Accessing Time (EAT)

we have $EAT(\text{effective access time}) = P * \text{TLB accessing time} + (1-P) * \text{miss memory time.}$

Where: P is Hit ratio.

$$\begin{aligned} \text{effective access time} &= 0.98 * 120 + 0.02 * 220 \\ &= 122 \text{ nanoseconds.} \end{aligned}$$

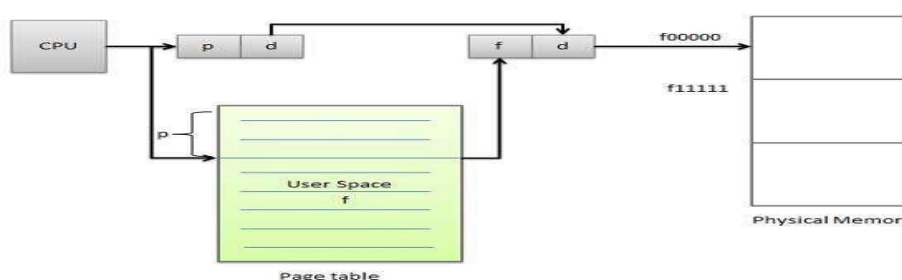


Fig: Paging Hardware

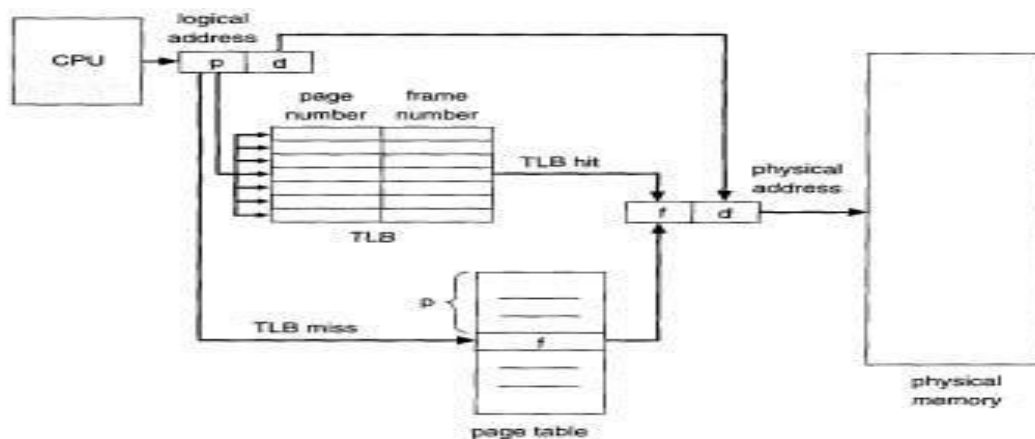


Fig: Paging Hardware with TLB

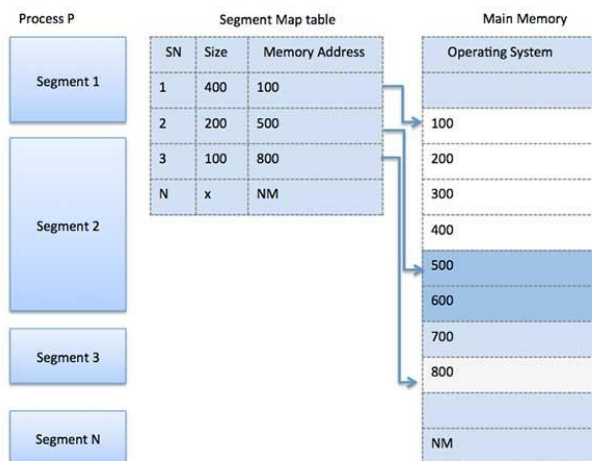
1.3 Segmentation

Segmentation is a memory management technique in which each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions. Each segment is actually a different logical address space of the program.

When a process is to be executed, its corresponding segments are loaded into non-contiguous memory though every segment is loaded into a contiguous block of available memory.

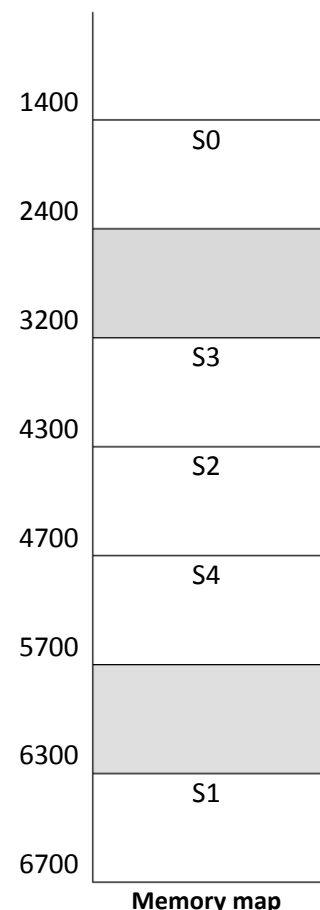
Segmentation memory management works very similar to paging but here segments are of variable-length where as in paging pages are of fixed size.

A program segment contains the program's main function, utility functions, data structures, and so on. The operating system maintains a **segment map table** for every process and a list of free memory blocks along with segment numbers, their size and corresponding memory locations in main memory. For each segment, the table stores the starting address of the segment and the length of the segment. A reference to a memory location includes a value that identifies a segment and an offset.



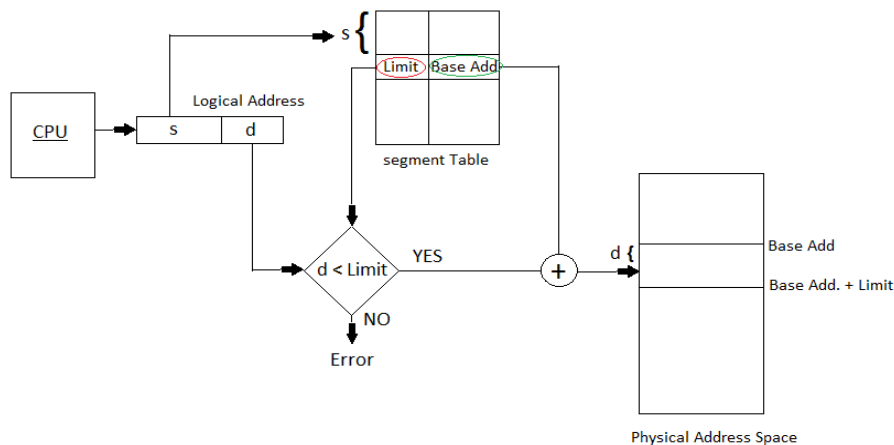
Example of Segmentation

Example 1: Given the following segment table, Provide its corresponding memory map.



Segment No.	Limit	Base address
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

Segmentation Hardware



Example 2: Consider the following segment table:

Segment No.	Base address	Limit
0	1219	700
1	2300	14
2	45	100
3	1327	580
4	1952	96

Calculate the physical addresses for the following logical addresses, and also determine which of the logical address will produce trap addressing error.

Logical Addresses: 0 – 430, 1 – 11, 2 – 100, 3 – 425, 4 – 95

The physical addresses can be calculated as follows:

- base address + logical address
- logical must be less than the limit
 - Segment 0: (430 < 700), valid, Physical address = (1219 + 430) = 1649
 - Segment 1: (11 < 14), valid, Physical address = (2300 + 11) = 2311

- Segment 2: $(100 \nless 100)$ **invalid; trap addressing error**
- Segment 3: $(425 < 580)$, valid, Physical address = $(1327 + 425)$ = 1752
- Segment 4: $(95 < 96)$, valid, Physical address = $(1952 + 95)$ = 2047

1.4 The Strength and Weakness of Memory Management Techniques

Technique	Description	Strengths	Weaknesses
Fixed Partitioning	Main memory is divided into a number of static partitions at system generation time. A process may be loaded into a partition of equal or greater size.	Simple to implement; little operating system overhead.	Inefficient use of memory due to internal fragmentation; maximum number of active processes is fixed.
Dynamic Partitioning	Partitions are created dynamically, so that each process is loaded into a partition of exactly the same size as that process.	No internal fragmentation; more efficient use of main memory.	Inefficient use of processor due to the need for compaction to counter external fragmentation
Simple Paging	Main memory is divided into a number of equal-size frames. Each process is divided into a number of equal-size pages of the same length as frames. A process is loaded by loading all of its pages into available, not necessarily contiguous, frames.	No external fragmentation.	A small amount of internal fragmentation
Simple Segmentation	Each process is divided into a number of segments. A process is loaded by loading all of its segments into dynamic partitions that need not be contiguous.	No internal fragmentation.	Improved memory utilization and reduced overhead compared to dynamic partitioning.
Virtual-Memory Paging	As with simple paging, except that it is not necessary to load all of the pages of a process. Nonresident pages that are needed are brought in later automatically.	No external fragmentation; higher degree of multiprogramming; large virtual address space.	Overhead of complex memory management
Virtual-Memory Segmentation	As with simple segmentation, except that it is not necessary to load all of the segments of a process. Nonresident segments that are needed are brought in later automatically.	No internal fragmentation, higher degree of multiprogramming; large virtual address space; protection and	Overhead of complex memory management.

Technique	Description	Strengths	Weaknesses
		sharing support.	